SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1407 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I SK.Rekha (192511023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
rekha

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i <= n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5}
   int total;

   total = sumArray(arr);

   if(total == 15) {
      printf("Correct Sum\n")
   }

   int a = 10, b = 0;
   int c = a / b;

   int *p;
   *p = 20;

   return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

Line no:	**error**	**description**
5	i <= n	Loop accesses one extra array element (arr[n]) causing out-bounds access.

12	Missing ;	Semicolon missing after array initialization.
15	sumArray(arr)	Function requires two arguments (arr and n), but only one is passed.
18	Missing ;	Semicolon missing after printf() statement.
22	a / b	Division by zero.
25	*p = 20	Pointer `p` is uninitialized before dereferencing.

(b) Classification of Errors (Understanding + Analysis)

ERROR	COMPILER PHASE
Missing ; after array declaration	Syntax error
Missing ; after printf()	Syntax error
Function call with missing argument	Semantic error
i <= n instead of i < n	Logical error
Division by zero	Runtime error
Dereferencing uninitialized pointer	Runtime error
Lexical Errors	none

(c) Compilation Behavior (Evaluation Level)

1. **Errors detected during compilation**

   Missing semicolon after array declaration.
   Missing semicolon after `printf()`.
   Incorrect function call (`sumArray(arr)`).

2. **Errors detected during execution**

   Array out-of-bounds (i <= n).
   Division by zero.
   Uninitialized pointer access.

(d) Code Correction (Application Level)

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;

   for(i = 0; i < n; i++) {
      sum += arr[i];
   }

   return sum;
}

int main() {
   int arr[5] = {1, 2, 3, 4, 5};
```

```
    int total;

    total = sumArray(arr, 5);

    if(total == 15) {
        printf("Correct Sum\n");
    }

    int a = 10, b = 2;
    int c = a / b;
    printf("Division Result = %d\n", c);

    int x = 20;
    int *p = &x;
    *p = 30;

    printf("Value of x = %d\n", x);

    return 0;
}
```

(e) Impact Analysis (Evaluation Level)

1. Improper Pointer Usage

- Dereferencing an uninitialized pointer may cause a **segmentation fault** or program crash.

2. Division by Zero

- Causes a **runtime exception**, leading to abnormal program termination.

3. Incorrect Loop Condition (i <= n)

- Accesses memory outside the array, producing **incorrect results** or causing a crash.

(f) Advanced Compiler Perspective (Create Level)

1. Improper Pointer Usage

Dereferencing an uninitialized pointer may cause a **segmentation fault** or program crash.

2. Division by Zero

Causes a **runtime exception**, leading to abnormal program termination.

3. Incorrect Loop Condition (i <= n)

Accesses memory outside the array, producing **incorrect results** or causing a crash.

Compiler Enhancements

1. **Static Code Analysis** to detect runtime issues before execution.
2. **Smart Error Recovery** so the compiler continues checking the remaining code and reports multiple errors in a single compilation.

Q2.The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
   int 1num = 100;
   float rate@ = 25.5;
   char ch = 'abc';
   int total = 50$20;
   int num#1 = 30;
   int %count = 40;

   printf("Welcome to Compiler Design);

   int x = 09;
   int y = 0xZ12;
   int z = 12.3.4;

   char str1[] = "Hello;
   char str2[] = "World";

   int _valid = 10;
   int invalid-id = 20;

   float value = 5..6;
   int data = 10@5;

   char c = 'xy';
   int 3value = 60;

   printf("Value: %d\n", total);

   return 0;
}
```

Tasks

(a) Error Identification

i) Identify all lexical errors in the program.

ii) List the line number and the invalid token.

(b) Error Classification

i) For each error, explain why it is considered a lexical error.

ii) Specify which rule of token formation is violated:

iii) Identifier rules

iv) Constant formation

v) String/character literal rules

vi) Operator/symbol rules

(c) Token Correction

i)Rewrite each invalid token into a valid token.

ii)Example: 2value → value2

(d) Compiler Behavior Analysis

i) Explain how a lexical analyzer (scanner) processes such errors.
ii)Discuss whether the compiler:
iii)Stops immediately
iv)Continues scanning using error recovery

(e) Advanced Thinking

i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.
ii) Suggest how error messages can be improved for beginners.

Q2.The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
   int 1num = 100;
   float rate@ = 25.5;
   char ch = 'abc';
   int total = 50$20;
   int num#1 = 30;
   int %count = 40;

   printf("Welcome to Compiler Design);

   int x = 09;
   int y = 0xZ12;
   int z = 12.3.4;

   char str1[] = "Hello;
   char str2[] = "World";

   int _valid = 10;
   int invalid-id = 20;

   float value = 5..6;
   int data = 10@5;

   char c = 'xy';
   int 3value = 60;

   printf("Value: %d\n", total);

   return 0;
}
```

Tasks

(a) Error Identification

LINE NO	INVALID TOKEN	ERROR
4	1num	Identifier starts with a digit.

5	rate@	@ is an illegal character in an identifier.
6	‘abc’	Character constant contains more than one character.
7	50$20	$ is an invalid symbol.
8	Num#1	# is an illegal character in an identifier.
9	%count	Identifier starts with %.
11	“welcome to compiler design)	Missing closing double quote (").
13	09	Invalid octal constant (9 is not allowed in octal).
14	0xz12	Z is not a valid hexadecimal digit.
15	12.3.4	Invalid floating-point constant.
17	“hello;	Missing closing double quote (").
20	Invalid-id	- is not allowed in identifiers.
22	5..6	invalid floating-point constant.
23	10@5	@ is an illegal symbol.
25	‘xy’	Character constant contains multiple characters.
26	3value	Identifier starts with a digit.

(b) Error Classification

Invalid Token	**Reason**	**Rule Violated**
1num, 3value	Identifier starts with a digit	Identifier Rule
rate@, num#1, %count, invalid-id	Contains illegal symbols	Identifier Rule
09, 0xZ12, 12.3.4, 5..6	Invalid numeric constants	Constant Formation
'abc', 'xy'	More than one character in character constant	Character Literal Rule
"Welcome to Compiler Design),	Missing closing quote	String Literal Rule
"Hello;		

50$20, 10@5	Illegal symbols used	Operator/Symbol Rule

(c) Token Correction

Invalid Token Correct Token

Invalid token	Correct token
1num	Num1
rate@	Rate
‘abc’	‘a’
50$20	50+20
Num#1	Num1
%count	Count
"Welcome to Compiler Design)	"Welcome to Compiler Design"
09	9
0xz121	0xA12
12.3.4	12.34
“hello;	"Hello"
Invalid-id	invalid_id
5..6	5.6
10@5	10+5
‘xy’	‘x’

(d) Compiler Behavior Analysis

i) How does the lexical analyzer handle these errors?

- The lexical analyzer scans the source code character by character.
- If it finds an invalid token, it reports a **lexical error** with the corresponding line number.

ii) Compiler behavior

- **Small errors:** The scanner may recover and continue scanning to find more errors.
- **Serious errors (e.g., unclosed string):** The compiler may stop or skip ahead until a valid token is found.

(e) Advanced Thinking

i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.

ii) Suggest how error messages can be improved for beginners.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

A. Lexical Errors
B. Syntax Errors
C. Semantic Errors
D. Intermediate Code Generation Issues (logic/precedence)
E. Code Optimization Issues (redundancy/inefficiency)

F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;

   int arr[4] = {1,2,3,4};
   printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;

   int total;
   total = x + y * result + 0;

   while(x < 20) {
      printf("%d\n", x);
   }

   return 0;
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

G. Lexical Errors
H. Syntax Errors
I. Semantic Errors
J. Intermediate Code Generation Issues (logic/precedence)
K. Code Optimization Issues (redundancy/inefficiency)
L. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;

   int arr[4] = {1,2,3,4};
   printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;

   int total;
   total = x + y * result + 0;

   while(x < 20) {
      printf("%d\n", x);
   }

   return 0;
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

I. Line No.	Error	II. Description
4	Missing ;	Semicolon missing after return a / b.
8	#value	Invalid identifier containing #.
9	Missing ;	Semicolon missing after float num = 15.5.
12	divide(10)	Function requires two arguments, but only one is passed.
14	if(result = 5)	Assignment (=) used instead of comparison (==).
15	Missing ;	Semicolon missing after printf().
20	x / y	Division by zero (y = 0).
23	arr[4]	Array index out of bounds (valid indices: 0–3).
26	*ptr = 25	Dereferencing an uninitialized pointer.
29	+ 0	Redundant addition; unnecessary expression.
31–33	while(x< 20)	Infinite loop because x is never incremented.

(b) Classify by Compiler Phase

Error	Compiler Phase
#value	**Lexical Error**
Missing semicolons	**Syntax Error**
divide(10)	**Semantic Error**
if(result = 5)	**Intermediate Code Issue** (wrong operator changes program logic)
total = x + y * result + 0	**Code Optimization Issue** (redundant + 0)
Division by zero	**Runtime Error**
Array out-of-bounds	**Runtime Error**
Uninitialized pointer	**Runtime Error**
Infinite loop	**Runtime Error (Logical Error)**

(c) Compilation vs Execution (Evaluation)

I. **Compile-time Errors**
   1)Invalid identifier (#value)
   2)Missing semicolons
   3)Incorrect function call (divide(10))

II. **Execution-time Errors**
   1)Division by zero

2)Array index out of bounds
3)Uninitialized pointer
4)Infinite loop

Why do some errors stop compilation?

Lexical and syntax errors prevent the compiler from understanding the program structure, so compilation stops before generating executable code.

(d) Correct the Program (Application)

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b;
}

int main() {
   int value = 20;
   float num = 15.5;
   int result;

   result = divide(10, 2);

   if(result == 5) {
      printf("Result is 5\n");
   }

   int x = 10;
   int y = 2;
   int z = x / y;

   int arr[4] = {1, 2, 3, 4};
   printf("%d\n", arr[3]);

   int data = 25;
   int *ptr = &data;
   *ptr = 30;

   int total;
   total = x + y * result;

   while(x < 20) {
      printf("%d\n", x);
      x++;
   }
```

```
    return 0;
}
```

(e) Deep Analysis (Evaluation)

I. **I. Incorrect Pointer Usage**

Dereferencing an uninitialized pointer may access invalid memory, causing a **segmentation fault** or crash.

II. Array Bounds Violation

Accessing memory outside the array can produce **incorrect values** or overwrite other memory, leading to unpredictable behavior.

III. Operator Precedence Error

Using = instead of == changes the program logic.

During intermediate code generation, the compiler generates assignment instructions instead of comparison instructions, producing incorrect execution.

(f) Advanced Task (Create Level)

I. Compiler Improvements

1. **Static analysis** to detect possible runtime errors (e.g., division by zero, null/uninitialized pointers) before execution.
2. **Improved debugging messages** with line numbers and suggested fixes (e.g., *"Did you mean == instead of =?"*).

II. Optimization Improvement

- The optimization phase can eliminate redundant expressions such as + 0, simplify constant expressions, and remove unnecessary calculations to generate more efficient code.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application	15%	Effectively applies	Applies concepts	Limited application;	Unable to

of Concepts		relevant concepts to analyze and resolve errors	with minor errors	requires guidance	apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand